

GHOST PREDATOR — Exit-Liquidity Logger (bid_log)

Research instrumentation: measure what a reversal kill-switch would ACTUALLY fill at, before anyone builds one · June 11, 2026

The Question We're Testing

Idea on the table: if the bot detects a hard reversal against a held position before the window closes, sell early and cut the loss ("secure profits"). Nobody should build this yet, because the entire question is the EXIT FILL: what do the BIDS for your token actually offer in the seconds when you'd want out? None of our bots record bid-side data — we all log asks (what we buy). This logger fixes that. **It writes a CSV and changes ZERO trading behavior.**

Why We Expect 'Sell on Reversal' to Fail (and want data to check)

1. **The exit reprices before you can act.** The bids for your token track the same Binance feed you watch; when your lead dies, bids collapse the same instant. Selling at fair odds minus spread, into the thinnest book of the window, is a systematic bleed unless your reversal detector beats the market's own repricing.
2. **Wobbles reverse back.** One live window crossed its own open SIX times in 5 minutes. A naive exit rule panic-sells winners. Our hold data: positions still leading at T-12s win ~96%.
3. **The best book on the team holds 100% to resolution** (310+ buys, ~zero early sells). If exits printed money, the benchmark account would probably be doing them.

The one variant worth testing: a CATASTROPHE-ONLY kill switch — sell only on a hard flip (e.g. ≥ 5 bps against, final seconds), never on wobbles. Whether it's +EV depends entirely on real bid depth at those moments. Hence: log first.

What To Capture (1 Hz, final 35s of every held position)

```
bid_log.csv columns:
ts_utc, pos_id, coin, side, secs_left, entry_price,
best_bid,                # top of the bid book for YOUR token
bid_depth_top5_usd,      # $ resting in the top 5 bid levels (your real exit capacity)
n_bid_levels, best_ask,
binance_move_bps        # your lead/deficit at that second (the would-be trigger)
```

Build It (two pieces, adapt names to your build)

1. **Stop discarding bids.** Your CLOB WSS feed already delivers them; most of our builds parse only asks. Mirror your ask handling:

```
bids = {} # token_id -> {bid_price: size}
# in the 'book' (snapshot) event: also parse ev['bids'] into bids[token]
# in 'price_change' events: side SELL -> asks dict, side BUY -> bids dict (was: skip BUY)
```

2. **A 1-second logger loop** over open positions inside their final 35s:

```
async def bidlog_loop():
    while True:
        for pos in open_positions(close_ts between now and now+35):
            bk = bids.get(pos.token_id, {})
            bb = max(bk) if bk else 0.0
            top5 = sorted(bk.items(), reverse=True)[:5]
            depth = sum(p*s for p, s in top5)
            csv_append(ts, pos.id, pos.coin, pos.side, secs_left, pos.entry,
                      bb, depth, len(bk), best_ask(pos.token_id), binance_move_bps(pos.coin))
        await asyncio.sleep(1.0)
```

Cost: ~36 CSV rows per held position; a few thousand rows/day. No new connections, no order paths touched.

The Analysis (after ~1 week, pooled across all 4 books)

Join bid_log to your resolved positions, then answer three questions:

1. **Detectability** — for each LOSS: did binance_move_bps flip hard (≥ 5 bps against entry side) at any second before T-5? (Our autopsy: ~half of losses flip by T-12 — detectable in principle.)
2. **Exit value** — at that first trigger second, what was best_bid x sellable depth, really? Exit proceeds vs the -100% you took by holding = the kill-switch's gross saving.
3. **False-alarm cost** — for each WIN: how often would the same trigger have fired? Those exits forfeit the win minus exit proceeds. Net EV = (2) summed over losses MINUS (3) summed over wins. Deploy only if clearly positive across MULTIPLE books — one book's positive result is noise until it replicates.

Four books x ~50 trades/day $\approx$ 1,400 instrumented positions per week — enough to decide for real.

Ghost Predator team note · instrumentation ONLY — nobody builds auto-sell until the pooled data says yes · no keys/infra in this doc · June 11, 2026